INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Fault Tolerance & Recovery

Recap

- Livelocks
- Domino Effect
- Communication induced checkpoints
- Coordinated / Un-coordinated ckeckpoints
- Message logging
- Juang-Venkatesan's algorithm
- Koo-Toueg's algorithm

Koo-Toueg Coordinated Checkpointing Algorithm

- The checkpoint algorithm takes two kinds of checkpoints on the stable storage:

  Permanent and tentative

- **Permanent checkpoint**: A local checkpoint at a process that is a part of a consistent global checkpoint.
- **Tentative checkpoint**: A temporary checkpoint that is made a permanent checkpoint on the successful termination of the checkpoint algorithm.
- In case of a failure, processes roll back only to their permanent checkpoints for recovery.

Improvement

- Control messages / Messages
- Let every outgoing message m has a field for a label (m.l)
- Each process uses monotonically increasing labels in its outgoing messages
- S = smallest label; L = largest label
- Let m be the last message that X recd from Y after X has taken its last permanent / tentative checkpoint.

  $last\_label\_recd_X[Y]$ = m.l if m exists; S otherwise
- Let m be the first message that X sent to Y after X took its last permanent / tentative checkpoint.

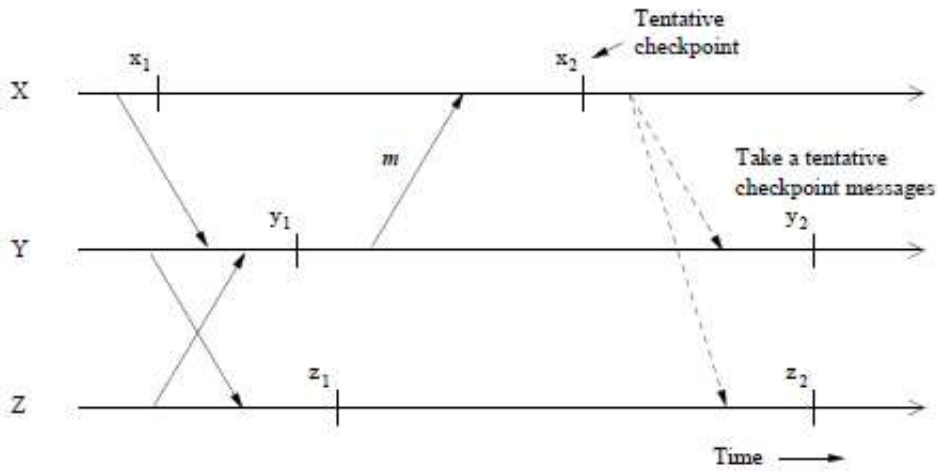
  $first\_label\_sent_X[Y]$ = m.l if m exists; S otherwise

- Whenever X requests Y to take a tentative checkpoint, X sends $last\_label\_recd_X[Y]$
- Y takes a checkpoint only if

  $last\_label\_recd_X[Y] \geq first\_label\_sent_Y[X] > S$
- Let $ckpt\_cohort_X$ be the set of all processes that should be asked to take checkpoints when X decides to take checkpoint
- $ckpt\_cohort_X = \{ Y \mid last\_label\_recd_X[Y] > S\}$

- Y takes a checkpoint only if

  $last\_label\_recd_X[Y] \geq first\_label\_sent_Y[X] > S$



The Rollback Recovery Algorithm

- A single process invokes the algorithm
- Checkpoint and the rollback recovery algorithms are not invoked concurrently.

First Phase

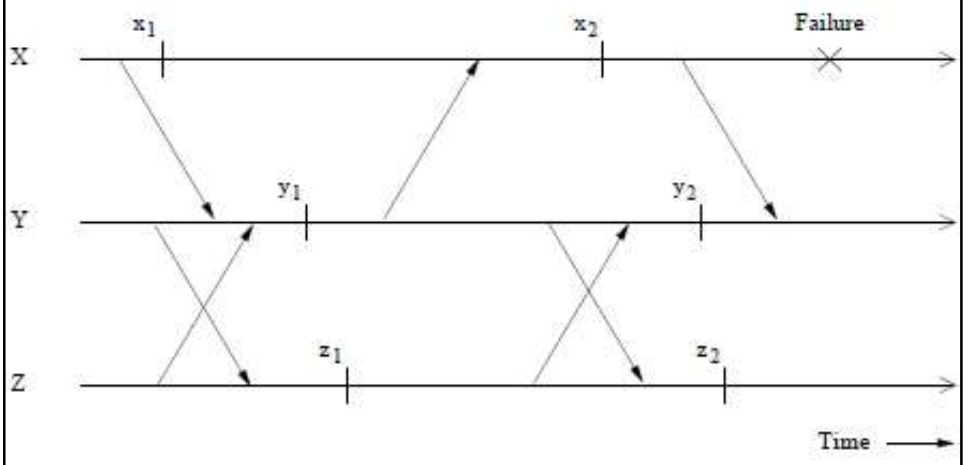
- An initiating process P_i sends a message to all other to check their willingness to restart from their previous checkpoints.
- A process may reply "no" due to any reason
- If all are willing to restart, P_i decides that all processes should roll back to their previous checkpoints.
- Otherwise, P_i aborts the roll back attempt

The Rollback Recovery Algorithm

Second Phase

- P_i propagates its decision to all the processes.
- On receiving P_i's decision, a process acts accordingly.
- **During the execution of the recovery algorithm, a process can not send messages related to the underlying computation while it is waiting for Pi's decision.**

The above recovery protocol causes all processes to roll back irrespective of whether a process needs to roll back or not



Improvement

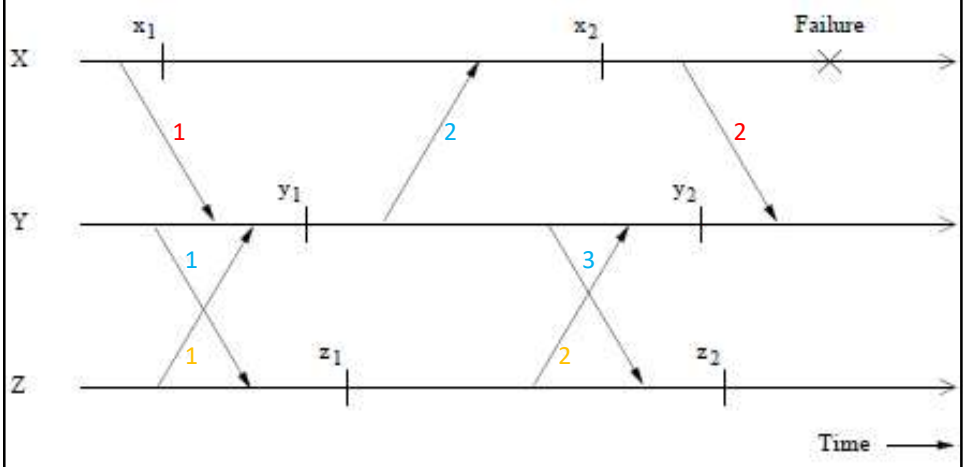
- Let us use labeling scheme used with checkpointing algo
- Let m be the last message that X sent to Y before X takes its last permanent checkpoint.

  $last\_label\_sent_X[Y]$ = m.l if m exists; L otherwise
- When X requests Y to restart from permanent checkpoint, X sends $last\_label\_sent_X[Y]$
- Y will restart only if

  $last\_label\_recd_Y[X] > last\_label\_sent_X[Y]$
- Also
- $roll\_cohort_X = \{ Y \mid X \text{ can send message to } Y\}$

- Y will restart only if

  $last\_label\_recd_Y[X] > last\_label\_sent_X[Y]$



Answer Yes / No

1. Is Koo-Toueg checkpointing method blocking?
2. In case of fault, faulty process always rolls back to permanent checkpoint but non faulty processes may rollback to permanent or tentative checkpoint.
3. Checkpoint and the rollback recovery algorithms can be invoked concurrently.
4. Is Koo-Toueg Rollback Recovery method blocking?

Fault Tolerance

- Use replication
- Commit protocols can be used to update multiple copies
- Multiple site failures / communication failures / network partitioning
- **Use voting** – Each replica is assigned some number of votes and a process needs to collect a certain number of votes before it can access a replica
- Can tolerate Multiple site failures / communication failures / network partitioning
- Static vs Dynamic voting mechanisms

Voting

System Model

- Replicas of files are stored at different sites
- Every file access operation requires that an **appropriate lock** is obtained
- Lock granting rule – "one writer and no readers" or "multiple readers and no writer"
- Every site has a lock manager
- Every file has a **version number**
- Version numbers are stored on stable storage and every successful write operation updates version number

Basic Idea

Read or write operation is permitted if a certain number of votes (read quorum (r) or write quorum (w)) are collected by the requesting process

Let **v be the total number of votes** assigned to all copies

r + w > v; w > v/2

(1) Send request for votes; (2) collect votes;

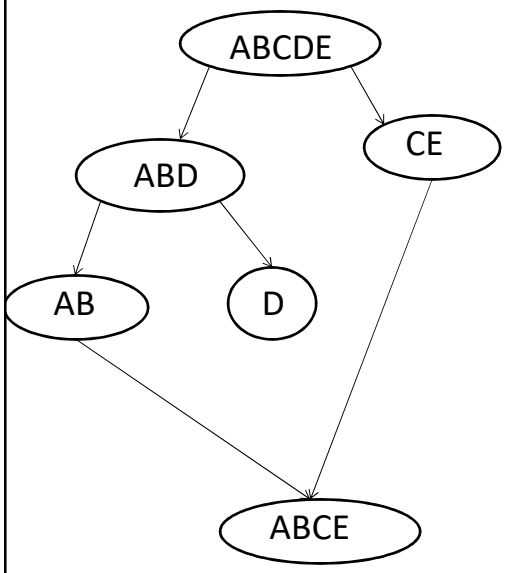
(3) perform operation if collected votes form quorum

Voting

- Can I assign more than one vote to a replica
- Does the geographical location of a replica matters
- Read / Write times
- How reliable is the replica

Dynamic Voting Protocols

- **Majority based approach**
- Dynamic vote reassignment



Majority Based Dynamic Voting Protocol

- Set of sites that can form a majority is dynamically changed
  - Sites that were updated when the most recent update was performed
- A **partition graph** is used to represent the history of the network's failure and recovery

The Protocol by Jajodia and Mutchler

Variables maintained by Replica *i*

- Version Number (VN*i*) – initially zero
- Number of Replicas updated (RU*i*) – initially N (Total replicas)
- Distinguished Sites List (DS*i*)

  RU*i* ≠ 3 and even: identifies replica that is greater than all other replicas that participated in the most recent update

  RU*i* ≠ 3 and odd: nil

  RU*i* = 3: lists all three replicas that participated in the most recent update

Distinguished Partition

- Site *i* performs update only if it belongs to distinguished partition

Distinguished Partition

- Site *i* has collected responses for its Vote_Request

  M = max{VNj : j ∈ P}; P is the set of responding sites

  Q = {j ∈ P : VNj = M}

  N = RU*j,* where j ∈ Q

- If Cardinality(Q) > N/2 => site i is member of distinguished partition
- If Cardinality(Q) = N/2; select a site j ∈ Q; if DSj ∈ Q => i is member of distinguished partition
- If N = 3 and if P contains two or more sites indicated by the DS variable of the sites in Q => i is member of distinguished partition

Distinguished Partition

Update

$VN_i = M + 1$

RU_i = Cardinality(P)

$$DS_i = \begin{cases} K \text{ if } RU_i \text{ is even, K is the site with hieghest order} \\ P \text{ if } RU_i = 3 \end{cases}$$

No change is made to RU_i and DS_i if static voting is used

The Protocol

- Site i issues a Lock_Request to its lock manager
- If (lock granted) send Vote_Request to all sites
- From all responses i decides whether it belongs to the **distinguished partition**
- If not in distinguished partition - Issue Release_lock to local lock manager and send abort to all sites in P
- If in distinguished partition – update local copy if current else get current copy and update
- Update VN*i, RUi, DSi*

- Site i sends a commit to all the participating sites along with the updates and values of VN*i, RUi and* DS*i*
- Issues a Release_lock to its lock manager

Site j

- When site *j* receives a Vote_Request, It issues a Lock_Request to its lock manager
- If (lock request is granted) return version number (VN*j*) RU*j* and DS*j* to site i
- Commit received: Perform update and update VN*j,* RU*j* and DS*j*
- Release_Lock received: release lock

- Five replicas of a file stored at site A, B, C, D and E
- All sites are accessible to every site (one partition)
- Following table shows the state of system

	A	B	C	D	E
VN	3	3	3	3	3
RU	5	5	5	5	5
DS	-	-	-	-	-

- B receives update and can communicate only with A and C

- B receives update and can communicate only with A and C

	A	B	C	D	E
VN	4	4	4	3	3
RU	3	3	3	5	5
DS	ABC	ABC	ABC	-	-

- C receives update and can communicate only with B

- C receives update and can communicate only with B

	A	B	C	D	E
VN	4	5	5	3	3
RU	3	3	3	5	5
DS	ABC	ABC	ABC	-	-

- D receives update and can communicate with B, C and E

- D receives update and can communicate with B, C and E

	A	B	C	D	E
VN	4	6	6	6	6
RU	3	4	4	4	4
DS	ABC	B	B	B	B

- C receives update and can communicate only with B

- C receives update and can communicate only with B

	A	B	C	D	E
VN	4	7	7	6	6
RU	3	2	2	4	4
DS	ABC	B	B	B	B

Consensus

- Consensus in case of replication
- We setup a collection of servers having replicated contents
  - To provide fault tolerance
  - For scaling
- All replicas must agree on the updates performed
  - Use a consensus algorithm
- All replicas must agree on the order updates have been performed
  - Choose a leader which will ensure in-order delivery to all updates
    - What if leader fails

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

PAXOS

Ref: Paxos Made Simple by Leslie Lamport

The Problem

- A collection of processes that can propose values.
- A consensus algorithm ensures that a single one among the proposed values is chosen.
- If no value is proposed, then no value should be chosen.
- If a value has been chosen, then processes should be able to learn the chosen value
- There are many cases where it is required
  - Mutual exclusion
  - Leader Election
  - Agreement on a common ordering of events

Requirements for consensus

Safety requirements:

- Only a value that has been proposed may be chosen,
- Only a single value is chosen, and
- A process never learns that a value has been chosen unless it actually has been.

Liveness requirements:

- ensure that some proposed value is eventually chosen and,
- if a value has been chosen, then a process can eventually learn the value.

PAXOS

- PAXOS was proposed by Leslie Lamport
- Paxos is a family of consensus algorithms

  (We will study basic PAXOS)
- Used in distributed systems to achieve agreement on a value, even in the presence of failures
- Many consensus algorithms are based on PASOX (Raft, Cheap-Paxos, Multi-Paxos)
- Industry softwares like Apache Zookeeper (the foundation of Apache Hadoop) & Kafka use consensus protocol based on Paxos

Applications using variants of Paxos

- **Google Chubby:** A highly available, distributed lock service.
- **Google Spanner:** A globally distributed, strongly consistent database
- **Google Megastore:** A storage system
- **Amazon DynamoDB:** Uses to ensure consistent replication
- **Microsoft Azure Storage:**
- **Apache Cassandra:** for its "lightweight transactions" (LWT) feature

PAXOS

- There are three classes of agents: *proposers*, *acceptors*, and *learners*.
- A single process may act as more than one agent
- Agents can communicate with one another by sending messages.
- Model: asynchronous, non-Byzantine, in which:
  - Agents operate at arbitrary speed, may fail by stopping, and may restart.
  - Since all agents may fail after a value is chosen and then restart, a solution is impossible unless some information can be remembered by an agent that has failed and restarted.
  - Messages can take arbitrarily long to be delivered, can be duplicated, and can be lost, but they are not corrupted.